

Like most programs, Macros consist of several components:

Functions — simply an operation which will occur; this may be as simple as an addition ($x + y$), or as complex as hundreds of lines of code. They have an input and an output.

Variables — values that are held in the computer's memory during the running of the function. Each variable must have a distinct name, so it can be uniquely referenced. They are defined by functions or statements, like so:

```
myName = "Chris";
```

or

```
myAge = 36;
```

Arguments — values that can be passed to a function to alter how it runs.

```
variable = function(arguments);
```

```
fPath = getDirectory("Choose a Directory");
```

**A lot of programming is about googling to find the answer you need!
You can't google if you don't know the right search terms though, so
here's some useful vocabulary:**

Indexing - the means by which we identify the specific item in an array or string that we would like to use. In ImageJ, indexing is in square brackets []:

```
array[0];
```

If we wanted to set the value of a variable to be equal to the value in a specific position in an array, we would use the following statement:

```
var = listOfNumbers[0];
```

Where var is the name of the newly-made variable, and listOfNumbers is the array containing the values we're interested in.

Note that if we have a variable that is equal to a whole number (an integer) we can index using that variable:

```
indexVariable = 1;
```

```
var = listOfNumbers[indexVariable];
```

Concatenating - joining two compatible variables together (arrays, strings, images)

Below, we concatenate three strings (two variable whose values are strings, and one character). The middle string is simply a space so that we don't have to include that character as part of either the first name or last name

```
firstName = "Chris";

lastName = "Law";

fullName = firstName + " " + lastName;
```

Below, we're adding an item onto the end of an existing list – I use this all the time in macro programming to make lists where I don't know what their length is going to be.

```
itemList = newArray("Beans", "Rice", "Pork", "Plantain");

tempItem = "Eggs";

itemList = Array.concat(itemList, tempItem);
```

Control Loop - FOR and WHILE loops. These enclose a series of commands that are repeated a number of times. FOR loops run for a set number of repetitions; WHILE loops run until a logical condition is evaluated as true or false Here, we've got a simple FOR loop to print out a list of organelles:

```
organelleList = newArray("Nucleus", "Golgi", "Rough ER", "Smooth ER", "Mitochondria", "Lysosomes");

for (organelle = 0; organelle < lengthOf(organelleList); organelle++){

    print(organelleList[organelle]);

}
```

In this loop, we give the iterative variable the name "organelle" – you should always pick a name for your iterative variable that makes sense. Or just choose a letter. Make sure that the

name used for the iterative variable is not used anywhere else!

In the parentheses at the start, we set 3 conditions:

organelle = 0 : this is the initial value of the iterative variable

organelle < lengthOf(organelleList) : this is the final value of the iterative variable; when this condition becomes false (i.e. the value of **organelle** becomes too big to index into the variable **organelleList**), the loop will end

organelle++ : this is what happens to the value of the iterative variable when the loop is completed; in this case it increases by 1

Everything within the curled parentheses are the commands that are repeated; here we're printing (i.e. showing the text) that exists in the position of the array **organelleList** that is defined by the iterative variable **organelle**.

Iterating - describes the behavior of control loops when they complete a loop and begin a new one. In the example above, we use ++ , which is the same as saying $x = x + 1$. You can also use - to move backwards through an array, or to count down.

Iterative variable - the variable whose condition is checked to determine the end of a FOR or WHILE control loop

Comments – bits of code that are ignored by the program; these allow you to make notes about what each bit of code is doing. These are really important. These are indicated by '//':

```
//This is a comment
```

String - a series of letters, usually enclosed by quotation marks " " . In imageJ, these are used to define arguments in many functions, as well as to prompt users, and, most importantly, to define the location of files on the computer.

Number - integers, decimals and floating points.

Array - a list of items, usually of the same type. The precise definition varies between programming languages. In imageJ, the tools to deal with them are generally prefaced by **Array**. e.g:

```
Array.concat(array1,array2);
```

Conditional Statements - IF/ELSE IF/ELSE statements. These set a condition that, if satisfied (or else, if not), results in a series of commands being performed. In imageJ, these come in the format:

```
if (condition == value){  
  
    commands;  
  
    }
```

Boolean Operators - '==', '!=', '<', '>', '<=', '>=' ; These check one variable against another (or a set value), and are usually used in conjunction with conditional statements.

```
if (lengthOf(itemList) >= 50){ //if there's 50 or more items  
  
    Array.slice(itemList,0,49); //just give me the first 50  
  
}
```

Escape Characters - hidden characters that control string behaviour; some examples from imageJ are:

\n - a new line

\t - a tab

\\ - a backslash (only the second backslash is shown)

Specific Hints & Tips:

Getting the contents of a folder:

```
fPath = getDirectory("Choose a Directory"); //prompts the user  
to point at a folder
```

```
fList = getFileList(fPath); //gets the list of everything in the  
folder
```

Be careful about how the operating system and FIJI list the files (i.e. how alphabetical order is determined)! Before you code anything else, use `Array.print(fList)` to show in which order the files are being read.

Getting information about images already open:

The two commands below query the active image about its name and folder of origin. They return `strings`.

```
name=getInfo("image.filename");
```

```
"example01.tif"
```

```
path=getInfo("image.directory");
```

```
"Users/Chris/Documents/"
```

These won't work if the image you are querying has been divorced from its original file (e.g. if it's a duplicate).

The command below pulls out the statistics for the pixels that make up the image:

```
getRowStatistics(nPixels, mean, min, max, std, histogram);
```

This differs from the command `getMinAndMax(min,max)`, which returns the min and max display values of the image (i.e. not the raw values of the pixels). If you want to use both commands in your macro, make sure you initialize new variables equal to min and max before using the other command, like so:

```
getRowStatistics(nPixels, mean, min, max, std, histogram);
```

```
minPxVal = min; //this stores the minimum pixel intensity
```

```
maxPxVal = max; //so you can use it later
```

```
getMinAndMax(min,max);
```

Verifying loop behavior

If your loops are not behaving as expected (looping infinitely, counting strangely, exiting early), you can check conditions at various points with the `print(variable)` and `Array.print(array)` commands; these will show the selected variables in the log, allowing you to verify that they are what you think they should be.

Indexing:

In Arrays, indexing is done by using square brackets [] **The first item in an array is numbered as 0** The last item in an array is numbered as `lengthOf(array)-1`

```
temp=newArray("A","B","C","D");
```

```
print(temp[0],temp[lengthOf(temp)-1]); //print the first and  
last items in the array
```


A, D

Initialising and filling arrays

Unlike some languages, making arrays in ImageJ requires the array to be initialized at the correct size first; you cannot assign a value to a location in an array that does not yet exist.

```
array=newArray(5); //make a new 5-item array
```

[0,0,0,0,0]

```
array[0]=256; //assign a value to the first item
```

[256,0,0,0,0]

```
array[(lengthOf(array)-1)]=4096; //assign to the last item
```

[256,0,0,0,4096]

```
array[5]=256; //assign to an item that doesn't exist
```

Error

Making measurements

You can make measurements in single ROIs using the following command:

```
roiManager("Measure");
```

To measure multiple rois, you can either use a loop similar to the following:

```
for(i=0,i<lengthOf(temp); i++){  
    roiManager("Select",i);
```

```
roiManager("Measure");  
  
}
```

Or use the multi-measure command (this works well for time-course stacks):

```
roiManager("Multi Measure ");
```

Be aware of the options that can be set for multi-measure; if you “append results”, it will bring back previously cleared results (see Common Pitfalls, below), so the first time you run it (i.e. when the iterative variable ==0), use it in the above form, and when i>0, use it in the form below:

```
roiManager("multi-measure measure_all one append");
```

Setting results

The results table in ImageJ is, frequently frustrating to use; but manipulating it is necessary if you want to measure anything that is not one of the options set in the Analyze>Set Measurements menu item (e.g. if you want to count the number of particles in a larger ROI). Use the following command to set a value in the results table to equal a variable:

```
setResult("Column", nResults-1, variable);
```

Use the following command to set a value in the results table to equal a variable:

```
getResult("Column", nResults-1);
```

nResults-1 sets/gets the value in the last row of the results table; setting this second parameter to nResults leads to the creation of a new row of data (or results in an indexing error, depending if you’re setting or getting results).

Common Pitfalls

- B&W reversed in binary images; this leads to failure to detect objects when using “Analyse particles...”

- Set the right colours at the top of the code using:

```
setBackgroundColour("Colour");
```

```
setForegroundColour("Colour");
```

- ROI management is important to avoid carry-over of ROIs from previous loops of the same code in a macro, or previous runs of the same macro.
- When you are finished with ROIs, delete them using the following lines

```
count = roiManager("Count");  
if (count>0){  
    roiManager("Deselect");  
    roiManager("Delete");  
}
```

- At the start of a macro, clear out previously-saved ROIs using the same code
- Clearing results is important for similar reasons as ROI management; if you want to make sure the results table is clear, you have to use the following 2 lines:

```
run("Clear Results");
```

```
updateResults();
```

- Familiarise yourself with escape characters; in Windows filepaths, “\” is used to denote levels of folders, and in the macro language, it becomes a hidden character, unless preceded by a 2nd “\”
- Mac and Windows file paths and filename handling are different; use the print command to understand how each of the operating systems orders files alphabetically.

- Saving in your “home” directory:
 - If you have included the **saveAs ()** command in your macro, but you don’t see the files appearing, look in the FIJI app package itself; your files probably got saved there
 - To avoid this, make sure to include the full file path in the **saveAs ()** command

Useful Websites:

<https://imagej.nih.gov/ij/developer/macro/functions.html>

http://imagej.net/Introduction_into_Macro_Programming
